A_

// AUDITNET · AUDITORÍA DE CONFIGURACIONES DE RED

Manual de instalación y desarrollo

Arquitectura, puesta en marcha, pruebas, despliegue en Azure y cómo extender la plataforma.

Versión 1.0 · octubre 2026
Español

Contenido

Introducción

Arquitectura

- Componentes
- Decisiones clave

Repositorio y convenciones

- Capas de cada servicio
- Convenciones

Instalación local

- Requisitos
- 1. Variables de entorno
- 2. Bases y servicios de apoyo
- 3. Servicios
- 4. Consola
- 5. Datos de demostración (opcional)

Pruebas

- Pruebas de los servicios
- Pruebas de pantalla
- En el CI

Cómo extender la plataforma

- Un endpoint nuevo
- Una migración
- Un tipo de regla de auditoría
- Una pantalla
- Una traducción
- Un correo
- Los manuales

Integración continua y despliegue en Azure

- GitHub Actions
- Recursos en Azure
- Crear todo desde cero
- Desplegar una versión a mano
- Seguridad del despliegue

Operación

- Modo demo y costos

Respaldos
Revisión periódica
Correo
Diagnóstico
Datos de demostración en Azure
Problemas reales y cómo se resolvieron
Permisos por perfil y dónde se controlan
Dónde se controla

Introducción

AuditNet es una plataforma web B2B para auditar configuraciones de equipos de red Cisco IOS/IOS-XE. Compara la configuración real de cada equipo (running-config) con una configuración esperada (baseline), detecta los desvíos, los prioriza por severidad y por la criticidad del equipo, genera alertas y reportes y conserva la evidencia de cada auditoría. Es multi organización: cada cliente ve solo sus datos.

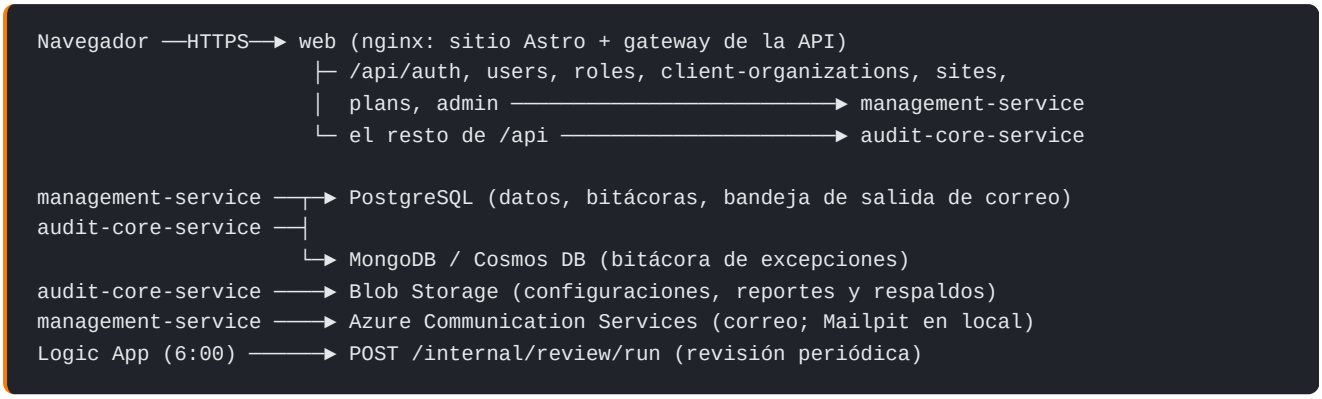
Este manual es para quien tiene que **instalar, operar o modificar** la plataforma: desarrolladores que se suman al proyecto y quien la despliega en Azure. Cubre:

- la arquitectura y por qué está armada así;
- cómo está organizado el repositorio y qué convenciones sigue;
- cómo levantarla en una computadora local y cargar datos de demostración;
- cómo correr las pruebas;
- cómo agregar un endpoint, una migración, una pantalla, una traducción o un correo;
- el circuito de integración y despliegue en Azure, y la operación diaria;
- los problemas reales que aparecieron durante el desarrollo y cómo se resolvieron;
- qué puede hacer cada perfil y dónde se controla en el código.

DATO	VALOR
Repositorio	`github.com/Leonel-Caruso/tfi-auditnet`
Rama de trabajo	`entrega-2` (se une a `main` al cerrar la entrega)
Decisiones de diseño	`docs/entrega-2/REGISTRO-DECISIONES.md` (D-01 en adelante)
Migraciones	`docs/database/MIGRACIONES.md`
Despliegue	`docs/deploy/AZURE.md`

> En los comandos, lo que va entre `<` y `>` se reemplaza por el valor real (sin los signos). Las contraseñas y secretos nunca se escriben en el repositorio ni en la línea de comandos: los scripts los piden al ejecutarse.

Arquitectura



› Componentes

COMPONENTE	TECNOLOGÍA	RESPONSABILIDAD
web	nginx + Astro 5 (sitio estático)	Sirve la consola y enruta `/api/...` a cada servicio en el mismo origen: los servicios no se exponen y no hace falta CORS. Publica la política de seguridad de contenido.
management-service (puerto 8081)	Java 21, Quarkus 3	Acceso y JWT, usuarios y perfiles, organizaciones y sedes, planes, correo, bitácora de sistema.
audit-core-service (puerto 8082)	Java 21, Quarkus 3	Dispositivos, configuraciones, baselines y reglas, motor de auditoría, hallazgos, alertas, tablero, reportes, revisión periódica, archivos y respaldos.
PostgreSQL 17	Flyway por servicio	Una base compartida; cada servicio migra sus tablas con su propia tabla de historial.
MongoDB / Cosmos DB	API de MongoDB 4.2	Bitácora de excepciones (la entidad NoSQL).
Blob Storage / Azurite	contenedor privado	Archivos con su SHA-256.

› Decisiones clave

- **Dos servicios y una base.** Se separan por responsabilidad (acceso y administración por un lado, auditoría por otro) y comparten PostgreSQL para que las consultas de alcance (organización, plan) sean simples. audit-core lee el plan con una consulta nativa; por eso management migra primero.
- **Seguridad.** El login devuelve un JWT firmado (HS256) con el usuario, su organización y sus perfiles. Cada endpoint declara los perfiles permitidos con `@RolesAllowed` y cada servicio aplica el alcance por organización. Cambiar el perfil, la organización o la contraseña invalida las sesiones abiertas.
- **Tres bitácoras.** Sistema (accesos y seguridad) y transacciones (valor anterior y nuevo de cada operación de negocio) en PostgreSQL; excepciones en MongoDB con su nivel configurable. Un código de seguimiento (`X-Request-ID` , lo pone nginx) relaciona una misma operación en las tres.

- **Evidencia inalterable.** Configuraciones, auditorías, seguimientos de hallazgos y reportes no se modifican: hay triggers de solo inserción y los reportes son una foto fija.
- **Correo con bandeja de salida.** Los avisos se guardan en `correos_salientes` dentro de la misma transacción del hecho que los origina y se despachan después, con reintentos. Así un fallo del proveedor no pierde avisos.
- **Multi idioma.** La consola traduce en el navegador con diccionarios; los mensajes de error se traducen con un catálogo según `Accept-Language`; los correos usan el idioma del destinatario.

Repositorio y convenciones

```
backend/management-service/   Quarkus: acceso, usuarios, organizaciones, planes, correo
backend/audit-core-service/   Quarkus: inventario, baselines, auditoría, hallazgos, reportes
frontend/web-app/             Astro: páginas, scripts, diccionarios, estilos y nginx
docker/                       PostgreSQL, MongoDB, Azurite y Mailpit para desarrollo local
infra/azure/                  Scripts de creación y operación en Azure (01 a 09)
scripts/                      load-env.ps1 y datos-demo/ (carga de demostración)
tests/pantallas/              Pruebas de pantalla (Playwright y API simulada)
tests/datos_demo/             Pruebas de los datos de demostración
tests/carga/                  Prueba de carga con k6
docs/                         Decisiones, migraciones, despliegue y manuales
.github/workflows/            CI, imágenes y despliegue
```

› Capas de cada servicio

```
ar/edu/uai/tfi/<servicio>/
  api/rest/         recursos JAX-RS y DTO: validan la entrada, @RolesAllowed y alcance
  application/      casos de uso y puertos (application/port), sin infraestructura
  domain/           modelo (records y enums) y repositorios como interfaces
  infrastructure/   adaptadores: Panache, seguridad, correo, bitácoras, archivos, i18n
```

Las dependencias van hacia adentro: `api → application → domain`, e `infrastructure` implementa los puertos del dominio y de la aplicación. Por eso los servicios de aplicación se prueban con implementaciones en memoria (`src/test/.../support`).

› Convenciones

- **Nombres en español** en el dominio, los servicios y los textos; las rutas de la API en inglés (`/api/devices`, `/api/findings`).
- **Código simple y defendible:** sin comentarios extensos; solo los que explican un porqué.
- **Migraciones Flyway** en `src/main/resources/db/migration` de cada servicio: `V<n>__<descripcion>.sql`. Nunca se modifica una migración ya aplicada; los cambios van en una nueva. Historial: `flyway_historial_management` y `flyway_historial_audit_core`.
- **Decisiones** en `REGISTRO-DECISIONES.md` con número `D-xx`: qué se decidió, por qué, alternativas y prueba.
- **Textos de la interfaz** nunca escritos en el HTML ni en los scripts: siempre desde los diccionarios (ver *Cómo extender la plataforma*).
- **Sin secretos en Git:** `.env` está ignorado; `.env.example` lista las variables sin valores.

Instalación local

› Requisitos

HERRAMIENTA	VERSIÓN
Docker Desktop	con Docker Compose
Java (JDK)	21
Maven	3.9 o superior
Node.js	24
Python	3.11 o superior (pruebas de pantalla, datos de demostración y manuales)
PowerShell	5 o superior (Windows); en Linux o macOS los mismos pasos con `bash`

› 1. Variables de entorno

Copiá `.env.example` a `.env` y completá los valores locales: base de datos, `JWT_SECRET`, el administrador inicial (`BOOTSTRAP_ADMIN_*`) y la organización inicial (`BOOTSTRAP_ORG_*`). `.env` no se sube a Git.

› 2. Bases y servicios de apoyo

```
.\scripts\load-env.ps1
docker compose --env-file .env -f docker/compose.database.yaml up -d
docker ps
```

Levanta `tfi-postgres` (5432), `tfi-mongo` (27017), `tfi-azurite` (10000, almacenamiento) y `tfi-mailpit` (los correos se leen en `http://localhost:8025`).

› 3. Servicios

En una terminal, **primero management-service** (aplica sus migraciones y crea los perfiles, la organización y el administrador inicial):

```
.\scripts\load-env.ps1
cd .\backend\management-service
mvn quarkus:dev
```

Cuando aparezca `Listening on: http://localhost:8081`, en otra terminal:

```
.\scripts\load-env.ps1
cd .\backend\audit-core-service
mvn quarkus:dev
```

› 4. Consola

```
cd .\frontend\web-app
npm install
npm run dev -- --host localhost
```

Abrí `http://localhost:4321` e ingresá con el administrador del `.env`. En desarrollo el navegador llama a los servicios en los puertos 8081 y 8082 (no pasa por nginx).

› 5. Datos de demostración (opcional)

`scripts/datos-demo` carga 6 organizaciones ficticias con sedes, usuarios, equipos, baselines, reglas y unas 100 auditorías, usando la API como lo haría un administrador:

```
python scripts\datos-demo\cargar.py --local --admin <usuario_admin> --correo
<casilla>@gmail.com
docker cp .\scripts\datos-demo\historia.sql tfi-postgres:/tmp/historia.sql
docker exec tfi-postgres sh -c 'psql -U $POSTGRES_USER -d $POSTGRES_DB -f
/tmp/historia.sql'
python scripts\datos-demo\cargar.py --local --admin <usuario_admin> --correo
<casilla>@gmail.com --reportes
```

- `cargar.py` pide la contraseña del administrador y la de los usuarios nuevos; no las guarda. Los correos de los usuarios son alias de la casilla (`casilla+usuario@dominio`) y sus avisos de auditoría quedan apagados.
- `historia.sql` reparte en las últimas 8 semanas las auditorías recién cargadas con sus hallazgos y alertas.
- `--reportes` genera los reportes una vez repartidas las fechas.
- Se puede volver a ejecutar: lo que ya existe se reutiliza.

Pruebas

QUÉ	COMANDO	DÓNDE
Unitarias e integración de cada servicio	`mvn test`	en `backend/<servicio>`
Tipos y diccionarios del frontend	`npm run check`	en `frontend/web-app`
Pruebas de pantalla	`npm run build:pruebas` y `python -m pytest tests\pantallas`	build en `frontend/web-app`, pytest desde la raíz
Datos de demostración	`python -m pytest tests\datos_demo`	raíz
Carga (k6)	ver `docs/entrega-2/CARGA.md`	raíz

› Pruebas de los servicios

Los servicios de aplicación se prueban sin base de datos, con repositorios en memoria (`support/EnMemoria...`) y un reloj fijo. Cada decisión del registro indica qué prueba la respalda. Antes de subir cambios, los dos servicios tienen que pasar `mvn test` completo.

› Pruebas de pantalla

`tests/pantallas` abre el sitio compilado en Chromium (Playwright) contra una API simulada (`servidor.py`, datos en `datos.py`) y recorre todas las pantallas con los cinco perfiles: que carguen sin errores, que no muestren textos técnicos, que entren en un celular de 390 px, que estén traducidas en inglés y portugués y que el tema oscuro no deje fondos claros.

```
cd .\frontend\web-app
npm run build:pruebas
cd ..\..
python -m pip install -r tests\pantallas\requirements.txt
python -m playwright install chromium
python -m pytest tests\pantallas
```

- `build:pruebas` compila el sitio para llamar a la API en el mismo origen, como detrás de nginx.
- El servidor de pruebas envía la misma política de seguridad de contenido que nginx: un script escrito dentro del HTML hace fallar la prueba. Por eso, para esperar una condición se usa `pagina.esperar("<expresión>")` y no `wait_for_function`.
- Para una pantalla nueva: sumar la ruta en `test_pantallas.py` y, si llama a endpoints nuevos, su respuesta en `RUTAS` de `servidor.py`.

› En el CI

Cada push a `entrega-2` o `main` corre, en GitHub Actions, los tests de los dos servicios, el chequeo y build del frontend, las pruebas de pantalla y las de datos de demostración. Si algo falla no se publican

imágenes.

Cómo extender la plataforma

› Un endpoint nuevo

1. **Dominio:** si hace falta, un record en `domain/model` y los métodos en la interfaz de `domain/repository`.
2. **Aplicación:** el caso de uso en un servicio de `application/<módulo>`. Valida, aplica el alcance de la organización y registra la operación en la bitácora de transacciones (`TrazabilidadPort`, con el valor anterior y el nuevo).
3. **Infraestructura:** la implementación del repositorio con Panache en `infrastructure/persistence`.
4. **API:** el recurso en `api/rest` con `@Path`, los DTO como records en `api/rest/dto` y `@RolesAllowed` con los perfiles que pueden usarlo.
5. **Permisos:** si es una acción nueva, sumarla a `MatrizPermisos` (management) y a `RolServiceTest`; la matriz se muestra en la pantalla de Usuarios.
6. **Gateway:** si el prefijo de la ruta es nuevo y corresponde a management-service, agregarlo a la expresión de `location ~ ^/api/(auth|users|...)` en `frontend/web-app/nginx/default.conf.template`. Lo demás va a audit-core.
7. **Mensajes:** cada mensaje de error nuevo que vea el usuario va en `src/main/resources/i18n/mensajes.txt` del servicio, con sus líneas `es:`, `en:` y `pt:` (si falta, se muestra en español).
8. **Pruebas:** del servicio con los repositorios en memoria y, si la usa una pantalla, la respuesta simulada en `tests/pantallas/servidor.py`.

› Una migración

1. Crear `V<n>__<descripcion>.sql` en `db/migration` del servicio dueño de la tabla, con el número siguiente al último de **ese** servicio.
2. Ajustar la entidad Panache y el repositorio.
3. Agregar la fila en `docs/database/MIGRACIONES.md`.
4. Al arrancar (`mvn quarkus:dev`, o el despliegue en Azure) Flyway la aplica sola. Nunca editar una ya aplicada.

› Un tipo de regla de auditoría

Los tipos son `DEBE_CONTENER`, `NO_DEBE_CONTENER` y `VALOR_ESPERADO`. Para sumar uno:

1. El valor en `TipoReglaConfiguracion` y su evaluación en `MotorAuditoriaService` (devuelve si cumple y la evidencia).
2. Una migración que amplíe el `CHECK` de `reglas_baseline.tipo`.
3. En la consola, la opción en el formulario de Reglas, su nombre en el diccionario (`condicion.<TIPO>`) y, si la evidencia tiene una frase nueva, su traducción en `lib/evidencia.ts`.
4. Pruebas del motor con configuraciones que cumplen y que no.

› Una pantalla

1. `src/pages/<ruta>.astro` con `MasterLayout` (el atributo `i18n` indica el prefijo de las claves del encabezado) y su script en `src/scripts/<ruta>.ts`, cargado con `<script src="../scripts/<ruta>.ts">`. No se escriben scripts dentro del HTML: la política de seguridad no los permite.
2. El ítem del menú en `components/Sidebar.astro` (código corto y perfiles) y la ruta en la lista de `components/Topbar.astro`; opcionalmente palabras para *Ir a módulo* en `layouts/MasterLayout.astro`.
3. Las claves `menu.<ruta>` y `ruta.<ruta>` en `i18n/es.ts`, `en.ts` y `pt.ts`, y un diccionario `i18n/pantallas/<pantalla>.ts` con `es`, `en` y `pt`, sumado en `i18n/index.ts`.
4. La ruta en `tests/pantallas/test_pantallas.py` (`PANTALLAS` o `PANTALLAS_POR_PERFIL`).

› Una traducción

- En el HTML: `data-i18n="clave"` para el texto y `data-i18n-placeholder`, `data-i18n-title`, `data-i18n-aria-label` o `data-i18n-label` para atributos.
- En los scripts: `t('clave', { dato })`, `plural(n, 'clave')` (claves `clave.uno` y `clave.otros`) y `etiqueta('grupo', codigo)` para códigos como severidades o estados.
- El tipo `Textos<typeof es>` obliga a que inglés y portugués tengan las mismas claves: `npm run check` falla si falta una. Los datos cargados por los usuarios (nombres de reglas, sedes) no se traducen.

› Un correo

Los textos están en `management-service/src/main/resources/correo/textos_<idioma>.properties` y el armado en `PlantillasCorreo`. Un tipo nuevo necesita su valor en `TipoCorreo`, sus textos en los tres idiomas y una prueba en `PlantillasCorreoTest`. Se encola con `CorreoService.encolar(...)` (o, desde audit-core, en la bandeja de salida común) y sale con los reintentos de siempre.

› Los manuales

Las fuentes están en `docs/manuales/<idioma>/usuario` y `docs/manuales/<idioma>/desarrollo`, en Markdown. Las capturas se sacan solas del sitio de pruebas con datos ficticios (`capturas.py`, `datos_manual.py`):

```
cd .\frontend\web-app; npm run build:pruebas; cd ..\..
python -m pip install -r docs\manuales\requirements.txt
python docs\manuales\generar.py
```

Los manuales de usuario quedan en `frontend/web-app/public/manuales` (la consola los ofrece para descargar) y este en `docs/manuales/pdf`.

Integración continua y despliegue en Azure

› GitHub Actions

`.github/workflows/imagenes.yml` corre en cada push a `entrega-2` o `main` que toque código, pruebas o el workflow:

TRABAJO	QUÉ HACE
Tests	`mvn test` de los dos servicios
Frontend	`npm ci`, `npm run check` y build
Pruebas de pantalla	Playwright contra el sitio compilado, y las de datos de demostración
Imágenes	Publica las 3 imágenes en GitHub Container Registry con las etiquetas `sha-<commit>` y la rama
Desplegar en Azure	Solo desde `entrega-2`: espera la **aprobación** del entorno `azure`, entra a Azure por OIDC (sin contraseñas), actualiza las 3 Container Apps a `sha-<commit>` y hace una prueba de humo

Para aprobar: *Actions → la ejecución → Review deployments → azure → Approve and deploy*.

› Recursos en Azure

RECURSO	NOMBRE	PARA QUÉ
Grupo de recursos	`rg-auditnet` (Brazil South)	Todo lo de la plataforma
PostgreSQL Flexible Server	`psql-auditnet-lcaruso`, base `tfi_auditnet`	Datos y bitácoras (B1ms, copias 7 días)
Container Apps	`ca-auditnet-web`, `ca-auditnet-management`, `ca-auditnet-core` en `cae-auditnet`	La consola con ingreso público y los dos servicios con ingreso interno
Cosmos DB para MongoDB	`cosmos-auditnet-lcaruso`	Bitácora de excepciones (plan gratuito)
Storage	`stauditnetlcaruso`, contenedor `auditnet-archivos`	Repositorio de archivos
Communication Services	dominio administrado por Azure	Correo (5 por minuto y 10 por hora)
Logic App	revisión diaria a las 6:00	Revisión periódica

› Crear todo desde cero

En **Azure Cloud Shell (Bash)**, con el repositorio clonado:

```
git clone -b entrega-2 https://github.com/Leonel-Caruso/tfi-auditnet.git
cd tfi-auditnet/infra/azure
bash 01-crear-base.sh            # grupo, PostgreSQL, base y entorno de Container Apps
bash 02-crear-apps.sh            # las 3 Container Apps desde las imágenes publicadas
bash 04-crear-cosmos.sh          # bitácora de excepciones
bash 05-crear-storage.sh         # repositorio de archivos
bash 06-configurar-github-oidc.sh  # despliegue desde GitHub sin contraseñas
bash 08-crear-email.sh           # correo
bash 09-crear-revision.sh        # revisión periódica
```

Los scripts piden las contraseñas al ejecutarse y las guardan como *secrets* de Container Apps; no hay secretos en el repositorio. Son reejecutables. `variables.sh` tiene los nombres de los recursos y registra los proveedores que hacen falta.

› Desplegar una versión a mano

```
cd ~/tfi-auditnet/infra/azure && bash 03-actualizar-imagenes.sh sha-<commit>
```

Cada servicio aplica sus migraciones al arrancar (management primero). El panel de administración muestra la etiqueta desplegada.

› Seguridad del despliegue

- HTTPS obligatorio con HSTS; nginx agrega `Content-Security-Policy`, `Permissions-Policy`, `X-Frame-Options` y `Referrer-Policy`.
- Los servicios no tienen ingreso público: solo se llega por nginx. `/internal` no se publica, salvo la revisión periódica, que exige un secreto de 256 bits.
- PostgreSQL acepta conexiones de servicios de Azure (incluido Cloud Shell). Para conectarse desde una PC hace falta una regla de firewall temporal con su IP.
- GitHub Actions solo puede actualizar las 3 apps, y cada despliegue requiere aprobación manual.

Operación

› Modo demo y costos

Las Container Apps escalan a cero cuando no hay tráfico: la primera solicitud tarda unos segundos en despertarlas. PostgreSQL puede quedar detenido para ahorrar crédito. Antes de una presentación:

```
cd ~/tfi-auditnet/infra/azure
bash 07-modo-demo.sh on       # 1 réplica mínima y PostgreSQL encendido
bash 07-modo-demo.sh estado
bash 07-modo-demo.sh off      # al terminar: vuelve a escalar a cero
```

El modo `on` consume la cuota aunque no haya tráfico. Los costos se controlan en *Cost Management* del portal.

› Respaldos

Desde el **Panel de administración → Generar respaldo**: un ZIP con los datos de todas las tablas, la versión del esquema y el guion de restauración, guardado en el repositorio de archivos. Azure conserva además copias automáticas de PostgreSQL por 7 días. Cómo restaurar: `docs/database/RESPALDOS.md`.

› Revisión periódica

Una Logic App llama todos los días a las 6:00 (hora de Argentina) a `POST /internal/review/run`. Por cada organización activa: re-audita los equipos cuya última auditoría superó el plazo, avisa los hallazgos vencidos y los equipos sin configuración nueva, y la primera revisión de cada mes envía el resumen del mes anterior. Se puede lanzar a mano desde el panel de administración. Si PostgreSQL estaba detenido, la ejecución de ese día falla y la siguiente se pone al día.

› Correo

Los avisos pasan por la bandeja de salida (`correos_salientes`); el panel de administración muestra los pendientes, enviados y fallidos, y **Enviar pendientes** fuerza el envío. El dominio administrado de Azure admite pocos envíos por hora: con muchas altas seguidas, los correos salen de a poco. En local se leen en Mailpit (`http://localhost:8025`).

› Diagnóstico

1. **Panel de administración:** estado de cada servicio y de sus componentes.
2. **Bitácoras:** buscar el código de seguimiento de la operación y filtrar las tres bitácoras con él.
3. **Registros de las apps:** `az containerapp logs show -g rg-auditnet -n ca-auditnet-management --tail 100` (o `ca-auditnet-core`, `ca-auditnet-web`).

› Datos de demostración en Azure

Desde Cloud Shell (el directorio no se conserva entre sesiones: clonar o actualizar el repositorio cada vez):

```
cd ~ && git clone -b entrega-2 https://github.com/Leonel-Caruso/tfi-auditnet.git; cd ~/tfi-auditnet && git pull
SITIO=https://$(az containerapp show -g rg-auditnet -n ca-auditnet-web --query properties.configuration.ingress.fqdn -o tsv)
python3 scripts/datos-demo/cargar.py --url $SITIO --admin <usuario_admin> --correo <casilla>@gmail.com
export PGPASSWORD=$(az containerapp secret show -g rg-auditnet -n ca-auditnet-management --secret-name db-password --query value -o tsv)
psql "host=psql-auditnet-lcaruso.postgres.database.azure.com dbname=tfi_auditnet user=auditnetadmin sslmode=require" -f scripts/datos-demo/historia.sql
unset PGPASSWORD
python3 scripts/datos-demo/cargar.py --url $SITIO --admin <usuario_admin> --correo <casilla>@gmail.com --reportes
```

Problemas reales y cómo se resolvieron

SÍNTOMA	CAUSA	SOLUCIÓN
`No plugin found for prefix 'quarkus'`	`mvn quarkus:dev` corrido desde la raíz del repositorio	Correrlo dentro de `backend/<servicio>`.
audit-core no arranca la primera vez	Sus migraciones referencian tablas de management	Levantar primero management-service.
El driver de MongoDB rechaza Cosmos DB	Azure creó la cuenta con la API 3.6 y el driver exige 4.2	Subirla desde el portal (*Upgrade MongoDB server version*); `04-crear-cosmos.sh` lo verifica.
`SubscriptionNotFound` al crear un recurso	El proveedor de ese tipo de recurso no estaba registrado en la suscripción	`variables.sh` los registra antes (`registrar_proveedores`).
`ExpressEnvironmentResourceNotSupported`	El entorno de Container Apps quedó en modo *express*, que no admite trabajos programados	La revisión periódica se programa con una Logic App (`09-crear-revision.sh`).
La primera pantalla tarda varios segundos	Las apps escalaron a cero o PostgreSQL está detenido	`07-modo-demo.sh on` antes de presentar.
Cloud Shell no encuentra el repositorio ni las variables	La sesión de Cloud Shell es efímera	Clonar o `git pull` y volver a definir las variables en cada sesión.
`unrecognized arguments` en `az postgres flexible-server firewall-rule`	La CLI cambió la sintaxis	Usar `--server-name <servidor> --name <regla>`. Desde Cloud Shell no hace falta regla: entra por la de servicios de Azure.
Los correos de bienvenida tardan en llegar	El dominio administrado de Azure limita a 10 envíos por hora	Esperar; la bandeja de salida los reintenta. Para más volumen, un dominio propio verificado.

SÍNTOMA	CAUSA	SOLUCIÓN
Un script del sitio no corre en Azure y sí en desarrollo	La política de seguridad bloquea scripts escritos dentro del HTML	Scripts en archivos aparte; Astro configurado con `assetsInlineLimit: 0`.
`wait_for_function` falla en las pruebas de pantalla	Evalúa texto como código y la política lo bloquea	Usar `pagina.esperar(...)`.
Playwright no encuentra el navegador	La versión instalada no coincide con la del paquete	`python -m playwright install chromium`.
La cuenta está bloqueada al probar un script	5 contraseñas equivocadas seguidas	Esperar 15 minutos o desbloquear desde Usuarios.
Después de desplegar se ve la versión anterior	El navegador guardó la página	Recargar con Ctrl+F5 (los archivos de `/_astro/` llevan hash; el HTML se revalida siempre).
Tu sesión terminó sin haber salido	Se cambió el perfil, la organización o la contraseña, o venció el token	Volver a ingresar: es el comportamiento esperado.

Permisos por perfil y dónde se controlan

MÓDULO · ACCIÓN	ADM	ORG	ANL	AUD	RSP
Tablero, estado del cliente, dispositivos, baselines, reglas, historial, hallazgos, alertas y reportes · consultar	✔	✔	✔	✔	✔
Dispositivos · registrar, modificar y deshabilitar	✔	✔			
Tipos de dispositivo · registrar	✔				
Baselines y reglas · crear, versionar y modificar	✔			✔	
Configuraciones · importar; auditorías · ejecutar	✔		✔	✔	
Hallazgos · cambiar el estado de seguimiento	✔		✔	✔	
Alertas críticas · derivar o devolver	✔		✔		✔
Reportes · generar	✔		✔	✔	
Organizaciones · registrar, modificar y deshabilitar	✔				
Sedes · registrar, modificar y deshabilitar	✔	✔			
Usuarios · gestionar administradores	✔				
Usuarios · gestionar perfiles de operación	✔	✔			
Plan · consultar las solicitudes	✔	✔			✔
Plan · solicitar un cambio		✔			
Planes · asignar, aprobar solicitudes y editar precios	✔				
Bitácoras y panel de administración	✔				

ADM: administrador del sistema · ORG: administrador de organización · ANL: analista de redes · AUD: auditor técnico · RSP: responsable de gestión IT.

› Dónde se controla

CAPA	DÓNDE	QUÉ CONTROLA
API	`@RolesAllowed` en cada recurso de `api/rest`	Qué perfiles pueden llamar a cada endpoint (403 si no).
Alcance	`UsuarioAutenticado` (audit-core), `Actor` y servicios de usuarios y sedes (management)	Que cada perfil, salvo el administrador del sistema, opere solo sobre su organización.
Plan	`ControlPlan` y `PlanService`	Límites de dispositivos y usuarios y el modo consulta del Piloto vencido.

CAPA	DÓNDE	QUÉ CONTROLA
Documentación	`MatrizPermisos` y `RolServiceTest`	La tabla que muestra la pantalla de Usuarios; hay que actualizarla si cambia un permiso.
Consola	`Sidebar.astro` (perfiles de cada ítem) y atributos `data-roles` / `data-admin-only`	Qué menús y botones se muestran. Es solo comodidad: la seguridad la aplica la API.
Pruebas	`test_perfiles.py`, `test_pantallas.py` y las pruebas de cada servicio	Que cada perfil vea y pueda hacer lo que le corresponde.